YBS 201 - Veri Yapıları ve Algoritmalar

Yığın (Stack) Kavramı

Doç. Dr. Ömer GÜLEÇ

omer@pau.edu.tr

www.pau.edu.tr/omer

Pamukkale Üniversitesi

İktisadi ve İdari Bilimler Fakültesi

Yönetim Bilişim Sistemleri



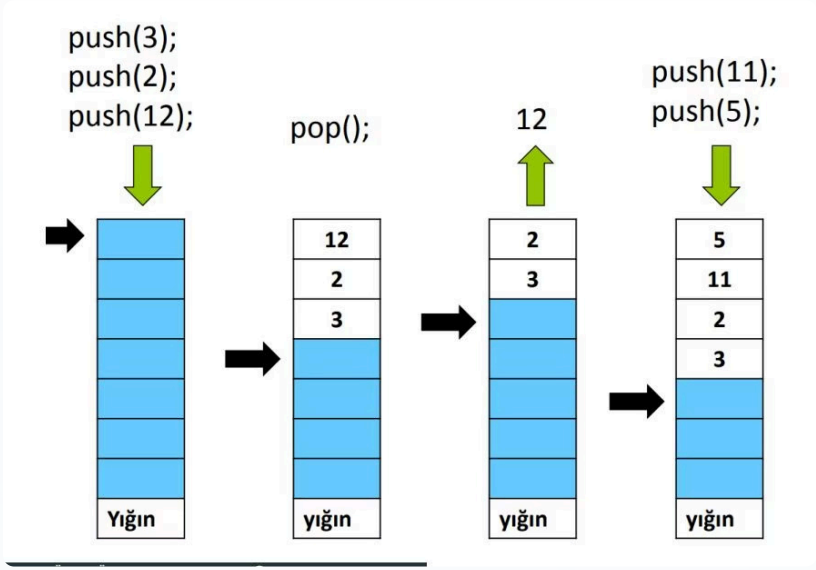
Yığın (Stack) Kavramı

FILO/LIFO Mantığı

First In Last Out ya da Last In First Out mantığıyla çalışan bellek üzerinde kurulmuş bir mekanizma

Yığın İşaretçisi

Stack pointer'e sahiptir ve herhangi bir ekleme yapılacağı zaman boş bir bellek odacığını gösterir



Yığın Temel İşlemleri

+

push()

Yeni bir nesne ekler

−

pop()

En son nesneyi çıkarır

→

top()

En son nesnenin yeri

↻

count()

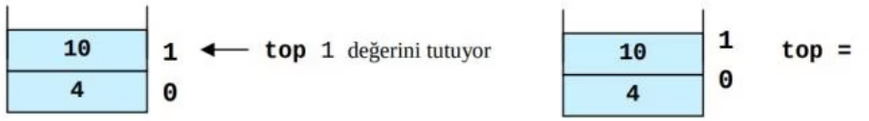
Depolanan nesne sayısı

∅

isEmpty()

Yığının boş olup olmadığı

```
void reset(stack *stk) {
    stk -> top =   ;
}
```



Yığın Kullanım Alanları

Gerçek Hayatta



Şarjör

Mermiler LIFO mantığıyla çıkar



Yemekhane Tepsileri

En üstteki tepsi ilk alınır



El Feneri Pilleri

Son takılan pil ilk çıkar

Bilgisayarda

- Programlarda UNDO/REDO işlemlerinde
- Web'de geri butonu
- Matematiksel işlemlerde
- Yazım denetimlerinde
- Infix : postfix dönüşümlerinde
- Mikroişlemciler / sensörlerin bellek yönetiminde
- Recursive fonksiyonların kullanımında

```
while (x == 7){
 printf("%d", a[2]);
}
```

Programlama Dillerinde ve Bellek Yönetiminde

Fonksiyon çağrıları (Call Stack): Bir fonksiyon çağrıldığında, parametreleri, lokal değişkenleri ve dönüş adresi stack'e eklenir. Fonksiyon tamamlandığında bu bilgiler stack'ten çıkarılır. → Örn: C, C++, Java, Python gibi tüm dillerde çalışır.

Bellek yönetimi: Programların çalışma zamanı (runtime) sırasında geçici verilerin tutulduğu stack alanı vardır. Bu sayede hızlı erişim sağlanır.

Algoritmalar ve Veri Yapıları

Geri izleme (Backtracking) algoritmaları: Labirent çözme, satranç hamleleri, Sudoku çözme gibi problemler stack ile uygulanır.

Derleyici tasarımı: Parantez dengeleme, ifadelerin çözümlemesi (infix → postfix/prefix dönüşümleri), sentaks kontrolü stack ile yapılır.

Derinlik öncelikli arama (DFS): Grafik ve ağaç algoritmalarında stack sıkça kullanılır.

İşletim Sistemleri

İş parçacığı (thread) yönetimi: Her iş parçacığının kendi stack alanı vardır. Bu alan, fonksiyon çağrıları ve yerel değişkenleri saklar.

Kesme (interrupt) işlemleri: CPU, bir kesme geldiğinde mevcut durumu stack'e kaydeder, kesme işlemini yapar, ardından stack'ten geri alarak devam eder.

Yazılım Mimarisi ve Araçlar

Undo/Redo işlemleri: Metin editörleri veya grafik programlarında geri al–ileri al işlevleri stack ile uygulanır.

Tarayıcı geçmişi: Web tarayıcılarında ileri/geri gezinme mantığı stack mantığına dayanır.

İşlem yığınları (Stack-based VM): Java Virtual Machine (JVM) veya .NET CLR gibi birçok sanal makine, stack tabanlı bir mimari ile çalışır.

Donanım ve Düşük Seviye Programlama

Assembly programlama: PUSH ve POP komutları ile stack kullanılır.

Mikrodenetleyiciler ve gömülü sistemler: Sınırlı bellek yönetimi için stack hayati önem taşır.

Stack ile Pratik Uygulamalar

Parantez Kontrolü

Stack kullanarak parantezlerin doğru eşleşip eşleşmediğini **nasıl** kontrol edebiliriz?

```python
def parantez_dengeli_mi(ifade):
    stack = []
    eslesmeler = {')': '(', ']': '[', '}': '{'}

    for ch in ifade:
        if ch in "([{":
            stack.append(ch)  # açılan parantezi stack'e ekle
        elif ch in ")]}":
            if not stack or stack[-1] != eslesmeler[ch]:
                return False
            stack.pop()  # eşleştiyse çıkar
    return not stack  # stack boşsa dengeli demektir

# Test
print(parantez_dengeli_mi("{ [ ( ) ] }"))   # True
print(parantez_dengeli_mi("{ ( [ ) ] }"))   # False
```

Stack ile Pratik Uygulamalar

Palindrom Kontrolü

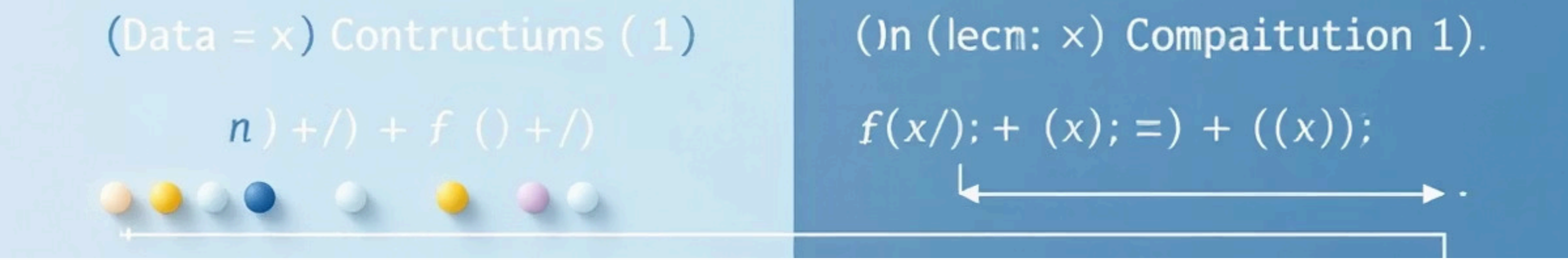
Stack kullanarak kelimelerin palindrom olup olmadığını **nasıl** kontrol edebiliriz?

Matematiksel İfadeler: x = a/b - c + d * e - a - c çözümü zor

a + b * c → (a + b) * c ? a + (b * c)

Infix zor: operatör önceliğine göre ayrıştırma, sıralama ve işleme gerekir. Bu nedenle prefix ya da postfix kullanılır.

Çoğu derleyici stack kullanarak prefix ve postfix'e çevirir. Operatörler: +, −, /, *, Operandlar: A, B, C...



Matematiksel Notasyon Türleri

Infix

Alışılagelmiş yaklaşım: 2 + 4 * 6

Operatörler operandlar arasına yerleştirilmiştir, parantez kullanımı gerekebilir

- (2 + 4) * 6 = 36
- 2 + (4 * 6) = 26

Prefix

Operatörler operandlardan önce

- +2*46 = +224 = 26
- *+246 = *66 = 36

Postfix

Önce operandlar daha sonra operatörler

- 246*+ : 224+ = 26
- 24+6* : 66* = 36

Basit Dönüşüm Örnekleri

1

Örnek 1

Infix: A + B

Prefix: +AB

Postfix: AB+

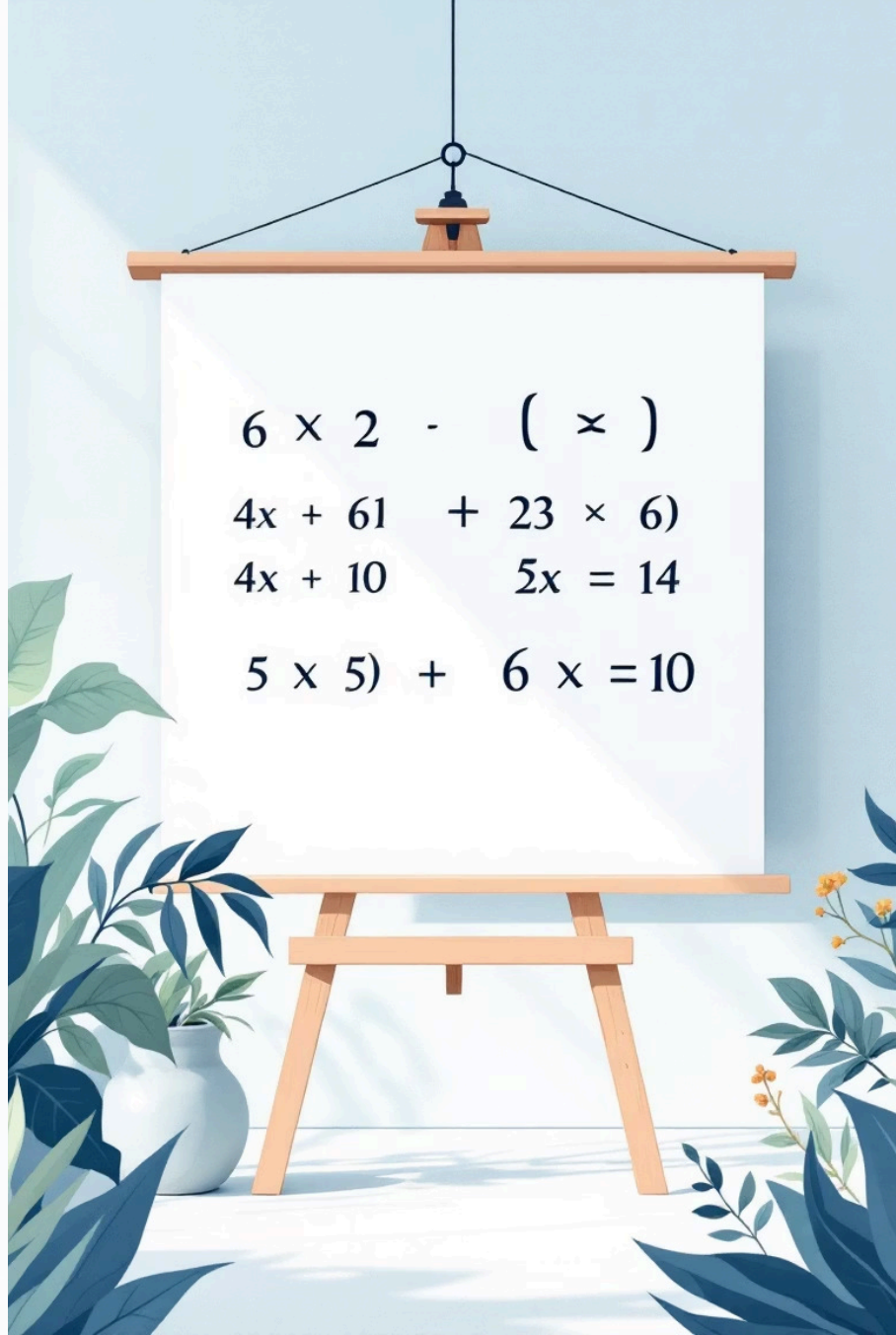
2

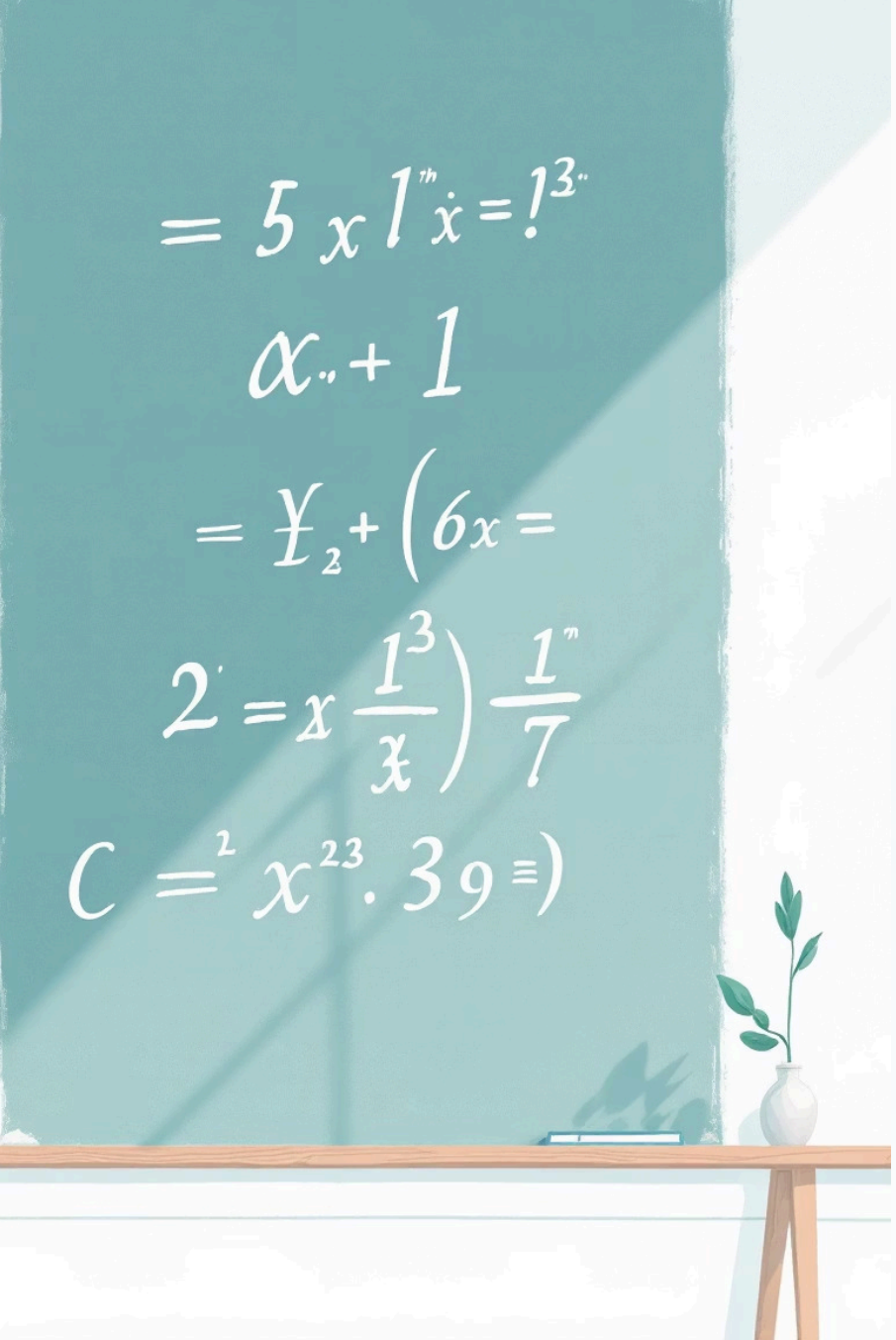
Örnek 2

Infix: A + B * C

Prefix: +A*BC

Postfix: ABC*+





Parantezli İfade Dönüşümleri

1

Örnek 1: (A + B) * C

Prefix: *+ABC

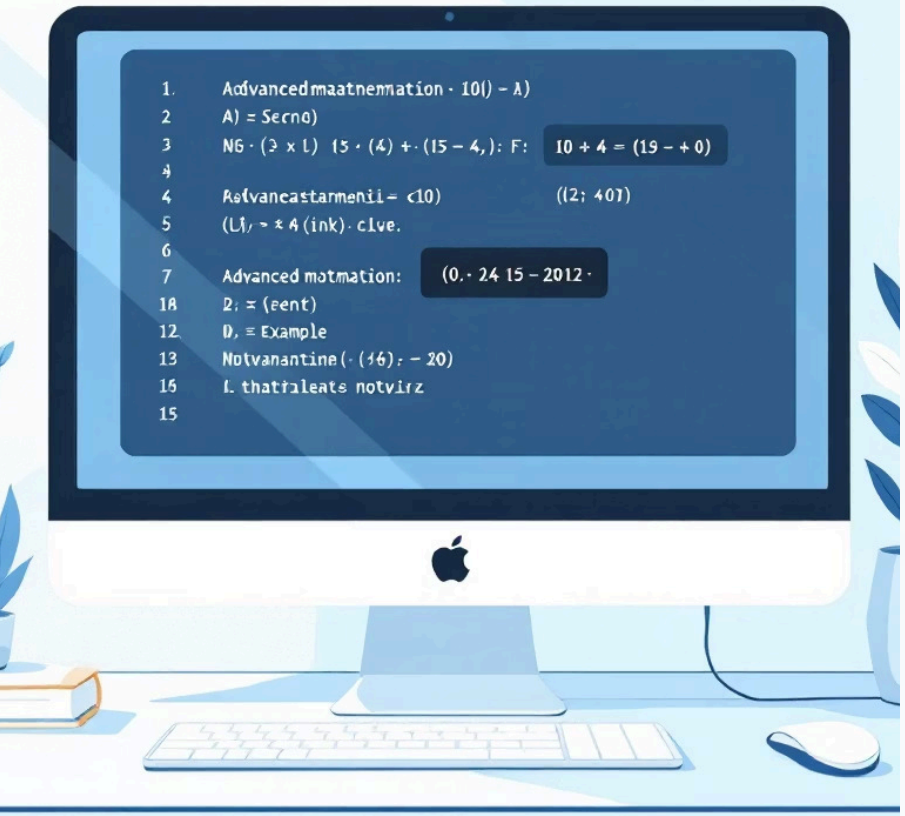
Postfix: AB+C*

2

Örnek 2: A + B * C + D

Prefix: ++A*BCD

Postfix: ABC*+D+



Karmaşık İfade Örnekleri

Örnek 1

Infix: (A + B) * (C + D)

Prefix: *+AB+CD

Postfix: AB+CD+*

Örnek 2

Infix: A * B + C * D

Prefix: +*AB*CD

Postfix: AB*CD*+

Örnek 3

Infix: A + B + C + D

Prefix: +++ABCD

Postfix: AB+C+D+

Son Örnek ve Soru

Karmaşık İfade

$$(X + Y) * ((A + B)/C)$$

Prefix: XY+AB+C/*

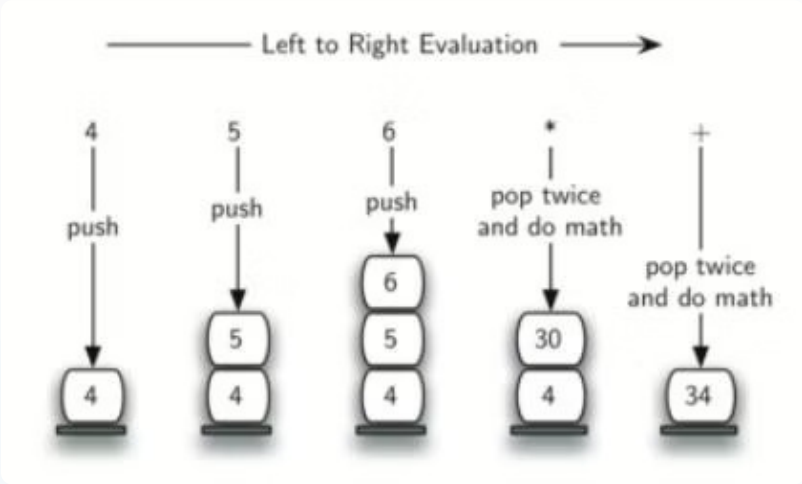
Postfix: *+XY/+ABC

Soru: Stack kullanarak nasıl infix, postfix çevrimi yapabiliriz?

Prefix: 456*+

Doç. Dr. Ömer GÜLEÇ
omer@pau.edu.tr
www.pau.edu.tr/omer

Son Örnek ve Soru



Doç. Dr. Ömer GÜLEÇ

omer@pau.edu.tr

www.pau.edu.tr/omer

YBS 201 - Veri Yapıları ve Algoritmalar